

PORTFOLIO PROJECT — STAGE 1 (FRANCE)

Stage 1 Report — Team Formation, Brainstorming & MVP

Project: CampOrga — a platform for managing coding & tech summer camps

School: Holberton School — France

Team: 2 people — Thélyaan Dufrénoy & Collins Odi Obi

Status: draft to be validated by the team before requesting QA review

1. Team Formation Overview

1.1 Team composition

The team was initially formed with 3 members. Harold NGuementa later stepped away from the project; the two remaining members redistributed the frontend responsibilities between them rather than replacing him. The table below details each member's technology ownership and proposed role.

Member	Framework	Proposed role
Thélyaan Dufrénoy	- React (+ Vite) - TypeScript - Tailwind CSS - TanStack Query (React Query) - React Hook Form - React Router - FastAPI - SQLAlchemy - Alembic - Pydantic - fastapi-users (or python-jose)	Fullstack Developer / Lead (incl. Frontend) <u>Languages</u> : Python, TypeScript, SQL. <u>Backend</u> : FastAPI (chosen Python framework), SQLAlchemy, Alembic, Pydantic. <u>Frontend</u> : React, TypeScript, Tailwind CSS, TanStack Query, React Hook Form, React Router. <u>Database</u> : PostgreSQL — schema design, queries. <u>Versioning</u> : Git / GitHub. <u>Coordination</u> : Discord, WhatsApp, shared documentation tool (Notion/Google Docs). <u>Others</u> : Docker (basic concepts, to collaborate with Collins on DevOps).
Collins Odi Obi	- React (+ Vite) - TypeScript - Tailwind CSS - FastAPI - SQLAlchemy - Docker - Docker Compose - Pytest - Vitest + React Testing Library - ESLint + Prettier - Ruff + Black	Backend / DevOps Developer (incl. Frontend) <u>Languages</u> : Python, SQL, TypeScript. <u>Backend</u> : FastAPI. <u>Frontend</u> : React, TypeScript, Tailwind CSS — shared UI development with Thélyaan. <u>Database</u> : PostgreSQL. <u>Containerization</u> : Docker, Docker Compose (to run API + DB together). <u>Backup (feature requested by the client)</u> : pg_dump for SQL export; a Python library (csv, pandas, or native PostgreSQL COPY) for CSV export. <u>Deployment</u> : A simple hosting platform like Railway or Render. <u>Quality</u> : Pytest, Vitest + React Testing Library, ESLint + Prettier, Ruff + Black. <u>Versioning</u> : Git / GitHub.

1.2 Why these roles?

The technical roles are defined from the stack chosen for CampOrga (React/TypeScript, Python, PostgreSQL, Docker) and the project's main areas of responsibility: the experience visible to the public (landing pages, registration), the business logic and data (API, database, backups), and overall coordination (planning, client relationship). With the team now at 2 members, frontend development is split between both roles rather than assigned to a dedicated third person.

- Fullstack Developer / Lead — manages the relationship with the client, ensures consistency between frontend and backend, handles cross-cutting tasks (data schema, architecture), and takes on part of the React/TS frontend work.
- Backend / DevOps Developer — develops the Python API, the PostgreSQL database, the backup feature (SQL/CSV), and Docker containerization, while also contributing to the React/TS frontend (landing pages, dashboard, forms).

A temporary Project Manager should be appointed at the first meeting to coordinate this first stage (organizing team check-ins, tracking deliverables); this role can rotate or be confirmed based on the team's preferences.

1.3 Communication and collaboration tools

- Discord — main coordination channel: technical discussions, progress updates, scheduled team check-ins.
- WhatsApp — quick, urgent messages between members.
- Centralized documentation — a shared space (Notion, Google Docs, or equivalent) to be chosen by the team to centralize this report and follow-up documents.
- Proposed norm: at least one weekly team check-in, decisions made by consensus, and any structuring decision (technical choice, scope change) documented in writing in the shared space.

1.4 Stakeholders

Stakeholder	Role	Impact on the project
External client	Project owner — provided the initial idea (a platform for managing summer camps), clarifies requirements, and validates deliverables (mockups, data schema...).	High impact — their feedback directly drives scope and design decisions.
Holberton mentors / QA	Assess whether the report and the project meet the Stage 1 learning objectives.	Medium impact — ensure the methodological quality of the deliverable.
Camp organizers (SEM, SWE, SSM, directors)	Internal users who will create and manage camps on the platform.	High impact — their needs define the dashboard and admin features.
Participants / visitors	Public end users who browse landing pages and register for camps.	Medium impact — their experience drives the registration rate, a key project goal.

2. Brainstorming and Idea Evaluation

This project's MVP was proposed by an external client (stakeholder): a platform for organizing summer camps. In line with the guidance for pre-assigned MVPs, the team's brainstorming work focused on adapting, refining, and scoping this idea rather than on fully open-ended idea generation.

2.1 Techniques used

- Mind Mapping — to visualize the possible features around the core idea (camp management, registrations, landing pages, user roles...) and how they connect.
- SCAMPER — to challenge the client's initial idea: Substitute (a generic nature theme for a clear "code & tech" positioning), Combine (organizer back-office + public landing page in a single tool), Eliminate (features out of scope for an MVP).
- "How Might We" questions — e.g. "How might we let several teams manage their camps without stepping on each other's toes?", "How might we make a visitor want to register within seconds?"

2.2 Ideas explored and reasons for rejection

Idea	Description	Reason for rejection
Generic multi-activity tool	A platform for any type of camp (sports, arts, nature, coding...).	Positioning too broad for an MVP; adds development complexity and dilutes the value proposition.
Parent-facing app only	A simple directory/search engine for camps aimed at families, with no organizer back-office.	Doesn't address the client's real need: internal, multi-team management is central to the request.
Native mobile app	An iOS/Android version instead of a web platform.	Development complexity and timeline too high for an MVP; the web stays accessible to everyone with no install.
"Nature / outdoor camp" theme	First direction for content and mockups, centered on nature discovery.	Rejected after client clarification: the camps actually focus on discovering code and computer science.

2.3 Evaluation criteria and ranking

Each idea was evaluated against four criteria: technical feasibility (skills already held by the team), impact for the client, alignment with the chosen tech stack, and scalability (ability to add features later, e.g. ML-based recommendations).

Idea	Feasibility	Impact	Technical alignment	Scalability
Coding camp platform (selected)	High	High	High	High
Generic multi-activity tool	Medium	Medium	Medium	Medium
Parent-facing app only	High	Low	Medium	Low
Native mobile app	Low	Medium	Low	Medium

3. Decision and Refinement of the Selected MVP

3.1 MVP summary

CampOrga is a web platform that lets several internal teams (SEM, SWE, SSM, campus and technical directors) create, publish, and manage summer camps focused on discovering code and computer science, while giving the public a dedicated landing page per camp to learn about it and register in a few clicks.

3.2 Problem it solves

Teams that organize coding summer camps currently have no centralized tool: each camp is managed separately, visibility across the organization's camps is poor, and the registration process for participants is neither standardized nor engaging.

3.3 Proposed solution

A single platform where organizers create a camp (dates, description, eligibility criteria) and automatically get a public landing page ready to be shared, while a super admin keeps an overview and manages access across the whole organization.

3.4 Target audience

- Internal organizers: SEM, SWE, SSM, campus and technical directors.
- Super admin: manages accounts and access to the platform.
- Participants (roughly 10-15 years old) and their parents: discover a camp through its landing page and register.

3.5 Type of application

A responsive web application (React / TypeScript on the frontend, Python API on the backend, PostgreSQL, containerized with Docker), accessible with no install, for both organizers (dashboard) and the public (landing pages).

3.6 Why this idea over the others

This concept scored best across the four evaluation criteria (see 2.3): it matches a real, already-scoped client request, uses technologies the team already knows or is learning (Python, SQL, React), and its modular structure (camps, registrations, landing pages, admin) allows features to be added later without reworking the core architecture.

3.7 Key objectives (SMART)

- Objective 1 — Let an organizer create a camp and publish its landing page in under 5 minutes, tested and validated before the end of the development sprint.
- Objective 2 — Let a visitor register for a camp in fewer than 3 steps (name, email, confirmation), to increase the landing page's conversion rate.
- Objective 3 — Give the super admin a consolidated view of all camps and users on the platform, with a working SQL/CSV backup export before final delivery.

3.8 Scope

In-Scope

- Camp creation and management by organizers (dates, description, eligibility criteria).
- Public landing page generated per camp, with countdown and LinkedIn sharing.
- Public registration (first name, last name, email, comment, confirmation).
- Organizer dashboard (managed camps, recent registrations).
- User and role management by the super admin, with SQL/CSV backup export.

Out-of-Scope (for this MVP)

- Online payment for registrations.
- Native mobile application.
- Advanced AI chatbot (a simple contact option is planned; a full conversational AI is deferred to a future iteration).
- Machine Learning-based camp recommendations.

3.9 Identified risks and mitigation

Risk	Mitigation
Reduced team size (2 members instead of 3) after Harold's departure	Redistribute frontend responsibilities between the Fullstack/Lead and Backend/DevOps roles (done); monitor workload closely and re-prioritize scope if needed.
Frequent back-and-forth with the client on scope and design (already observed: camp theme, visual style)	Document every client feedback in writing and get it validated before starting a new design or development iteration.
Team members at different skill levels in React/TypeScript	Plan dedicated learning time early in the project and assign tasks based on the strengths identified in Section 1.1.
Tight delivery deadline (first week of September)	Strictly prioritize in-scope features and push any non-essential addition to a future iteration.

4. Summary

The team (2 members: Thélyaan Dufrénoy — Fullstack/Lead, and Collins Odi Obi — Backend/DevOps, both sharing frontend development — see Section 1) coordinates via Discord and WhatsApp, with a weekly check-in and decisions made by consensus. The MVP was provided by an external client, so the team's ideation work focused on adapting and scoping this idea rather than creating it from scratch, using Mind Mapping, SCAMPER, and "How Might We" questions.

The selected concept, CampOrga, is a web platform that lets several internal teams create and manage summer camps focused on code and computer science, with a public landing page per camp to maximize registrations. It was preferred over the other ideas explored (generic tool, parent-only app, native mobile app) because it directly matches the client's stated need, builds on skills already present in the team, and leaves room for future evolutions (AI-based recommendations, mobile app) without calling the MVP's architecture into question.

Once the team information (Section 1.1) is completed, this report constitutes the documented decision-making process required for Stage 1 and serves as the foundation for the following planning phase.